Logbook Kegiatan — 24 April 2026

Penelitian	Sistem Kendali Drone Kamikaze Berbasis Deteksi Objek Warna dalam Simulasi HITL
Tim	Musa El Hanafi & Muhammad Ihsan Fahriansyah
Lokasi	Lab Komputer SMA Swasta Alfa Centauri, Kota Bandung
Hari/Tanggal	Jumat, 24 April 2026

Kegiatan hari ini berfokus pada pengembangan modul **drone-seeker** — komponen computer vision yang bertanggung jawab mendeteksi dan melacak target berwarna untuk sistem kendali drone kamikaze. Kegiatan dibagi dalam empat bagian: **Bagian 1** pengenalan OpenCV dan konsep dasar ruang warna; **Bagian 2** implementasi aplikasi dasar (`imshow` dan konversi BGR→HSV); **Bagian 3** implementasi pipeline deteksi dan pelacakan objek berdasarkan warna; **Bagian 4** implementasi aplikasi kalibrasi warna interaktif menggunakan trackbar.

Arsitektur Modul drone-seeker

Modul `drone-seeker` bertugas memproses frame kamera secara real-time, mendeteksi target berwarna, dan mengirimkan perintah kendali ke Pixhawk via MAVLink. OpenCV menjadi inti pipeline pemrosesan visual.

```
Kamera / Video Feed
       |
       ▼
  Frame Capture (VideoCapture)
       |
       ▼
  Konversi BGR → HSV
       |
       ▼
  Masking berdasarkan rentang Hue/Sat/Val
       |
       ▼
  Morphological Filtering (erode + dilate)
       |
       ▼
  Contour Detection → Centroid Kalkulasi
       |
       ▼
  Error Kalkulasi (ex, ey dari pusat frame)
       |
       ▼
  MAVLink Command (velocity setpoint / attitude)
       |
       ▼
      Pixhawk (HITL via PPP)
```

Bagian 1 — Mengenal OpenCV

1. Pengenalan OpenCV

Kegiatan:
Mempelajari OpenCV sebagai library computer vision utama yang digunakan dalam modul `drone-seeker` untuk deteksi dan pelacakan target berwarna.

Apa itu OpenCV:

OpenCV (Open Source Computer Vision Library) adalah library open-source untuk pemrosesan gambar dan video secara real-time. Library ini mendukung Python, C++, dan Java, serta berjalan di Linux, macOS, dan Windows.

Aspek	Keterangan
Nama lengkap	Open Source Computer Vision Library
Versi yang digunakan	OpenCV 4.x
Bahasa	Python 3 (binding `cv2`)
Lisensi	Apache 2.0
Website	https://opencv.org

Konsep Dasar yang Dipelajari:

Konsep	Penjelasan
Image sebagai array NumPy	Gambar direpresentasikan sebagai `ndarray` dengan shape `(H, W, 3)` untuk BGR atau `(H, W)` untuk grayscale
Ruang warna BGR	OpenCV menggunakan urutan Blue-Green-Red (bukan RGB) secara default
Ruang warna HSV	Hue-Saturation-Value — lebih robust untuk segmentasi warna di bawah perubahan pencahayaan
Masking	Binary image hasil threshold yang digunakan untuk mengisolasi piksel target
Contour	Kurva yang menghubungkan piksel dengan intensitas sama — digunakan untuk menemukan batas objek

Mengapa HSV lebih baik dari BGR untuk deteksi warna:

Kondisi	BGR	HSV
Cahaya redup	Semua channel turun — warna berubah	Hue tetap stabil, hanya Value turun
Bayangan	Warna menjadi gelap dan tidak konsisten	Hue tidak berubah, Saturation sedikit turun
Sorotan lampu	Channel meledak ke nilai tinggi	Value naik, Hue tetap bisa di-mask
Implementasi	Sulit — 3 threshold per warna	Mudah — 1 range Hue + toleransi Sat/Val

Instalasi:

```
pip install opencv-python numpy
```

Hasil: Konsep dasar OpenCV, representasi gambar sebagai array NumPy, dan keunggulan ruang warna HSV untuk deteksi warna dipahami.

Bagian 2 — Aplikasi Dasar: Imshow dan BGR→HSV

---

2. Aplikasi Imshow

Kegiatan:
Membuat aplikasi pertama menggunakan OpenCV untuk membuka gambar, menampilkannya di layar, dan mempelajari cara kerja window OpenCV.

Kode (`app_imshow.py`):

```
import cv2

# Baca gambar dari file
img = cv2.imread("target.png")

if img is None:
    print("Error: gambar tidak ditemukan")
    exit()

# Tampilkan info gambar
print(f"Shape : {img.shape}")   # (height, width, channels)
print(f"Dtype : {img.dtype}")   # uint8
print(f"Ukuran: {img.size} piksel")

# Tampilkan gambar dalam window
cv2.imshow("Gambar Target", img)

# Tunggu input keyboard
# 0    = tunggu selamanya
# 1000 = tunggu 1 detik (millisecond)
key = cv2.waitKey(0)

if key == ord('s'):
    cv2.imwrite("output.png", img)
    print("Gambar disimpan sebagai output.png")

# Tutup semua window
cv2.destroyAllWindows()
```

Penjelasan fungsi utama:

Fungsi	Parameter	Keterangan
`cv2.imread(path)`	path: str	Membaca gambar → ndarray BGR, `None` jika gagal
`cv2.imshow(name, img)`	name: str, img: ndarray	Menampilkan gambar dalam named window
`cv2.waitKey(delay)`	delay: int (ms)	Menunggu input keyboard; 0 = tunggu selamanya
`cv2.destroyAllWindows()`	—	Menutup semua window OpenCV
`cv2.imwrite(path, img)`	path: str, img: ndarray	Menyimpan gambar ke file

Menampilkan feed kamera langsung (webcam):

```
import cv2

cap = cv2.VideoCapture(0)  # 0 = kamera default

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.imshow("Kamera", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break

cap.release()
cv2.destroyAllWindows()
```

Hasil: Aplikasi imshow berhasil menampilkan gambar statis dan feed kamera real-time.

---

3. Aplikasi Konversi BGR → HSV

Kegiatan:
Membuat aplikasi untuk memvisualisasikan perbedaan antara ruang warna BGR dan HSV, sekaligus memahami cara kerja `cv2.cvtColor`.

Kode (`app_bgr2hsv.py`):

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Konversi BGR → HSV
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Pisahkan channel HSV
    h, s, v = cv2.split(hsv)

    # Pisahkan channel BGR
    b, g, r = cv2.split(frame)

    # Buat visualisasi: channel H, S, V sebagai grayscale
    # dan channel B, G, R sebagai grayscale
    vis_top = np.hstack([frame, hsv])        # BGR asli | HSV (tampilan warna)
    vis_mid = np.hstack([
        cv2.cvtColor(b, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(g, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(r, cv2.COLOR_GRAY2BGR),
    ])  # Channel B | G | R
    vis_bot = np.hstack([
        cv2.cvtColor(h, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(s, cv2.COLOR_GRAY2BGR),
        cv2.cvtColor(v, cv2.COLOR_GRAY2BGR),
    ])  # Channel H | S | V

    # Resize agar muat di layar
    def half(img):
        return cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.imshow("BGR | HSV", half(vis_top))
    cv2.imshow("B | G | R", half(vis_mid))
    cv2.imshow("H | S | V", half(vis_bot))
```

```
    # Cetak nilai piksel di titik tengah
    cy, cx = frame.shape[0] // 2, frame.shape[1] // 2
    bgr_val = frame[cy, cx]
    hsv_val = hsv[cy, cx]
    print(f"BGR={bgr_val} → HSV={hsv_val}", end="\r")

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Rentang nilai HSV di OpenCV:

Channel	Rentang OpenCV	Keterangan
H (Hue)	0 – 179	OpenCV menggunakan setengah dari 360°
S (Saturation)	0 – 255	0 = putih/abu-abu, 255 = warna penuh
V (Value)	0 – 255	0 = hitam, 255 = terang penuh

> **Penting:** Hue di OpenCV = Hue_sebenarnya / 2. Merah = 0–10 dan 170–179 (melingkar).

Referensi Hue untuk warna umum:

Warna	H (OpenCV)	H (360°)
Merah	0–10 dan 170–179	0° dan 350–360°
Oranye	10–25	20–50°
Kuning	25–35	50–70°
Hijau	35–85	70–170°
Cyan	85–100	170–200°
Biru	100–130	200–260°
Ungu/Magenta	130–170	260–340°

Hasil: Aplikasi BGR→HSV berhasil menampilkan visualisasi semua channel secara bersamaan. Pemahaman tentang rentang nilai HSV di OpenCV tercapai.

Bagian 3 — Aplikasi Deteksi dan Pelacakan Objek Berdasarkan Warna

---

4. Pipeline Deteksi Objek Berwarna

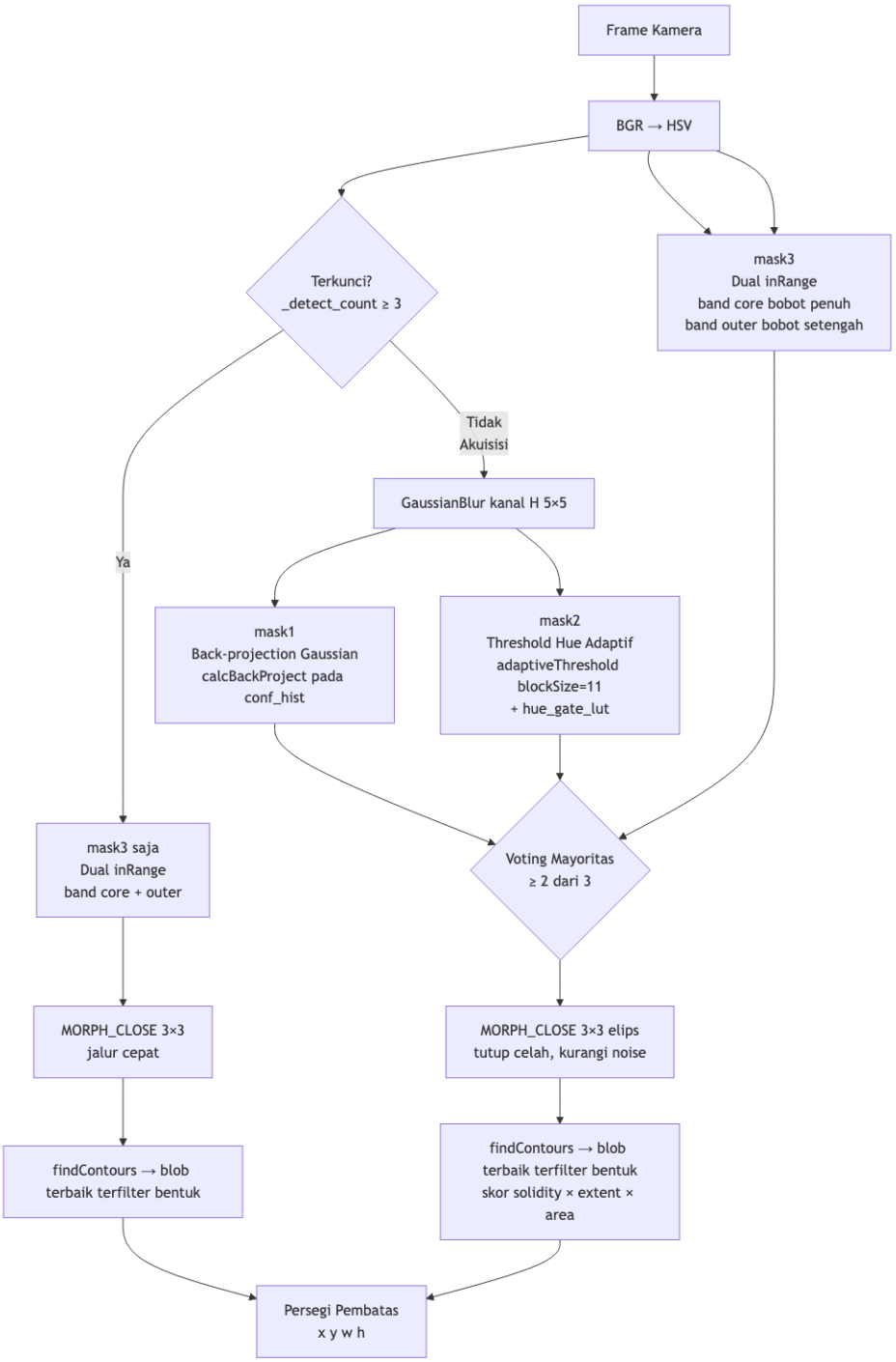
Kegiatan:
Membangun pipeline lengkap untuk mendeteksi objek berdasarkan warna dalam video real-time, menampilkan bounding box dan centroid objek yang terdeteksi.

Kode (`app_color_tracker.py`):

```
import cv2
import numpy as np

# --- Konfigurasi warna target (contoh: oranye) ---
# Rentang HSV untuk warna oranye
HSV_LOWER = np.array([10, 120, 100])
```

```
HSV_UPPER = np.array([25, 255, 255])

# Ukuran minimum contour yang dianggap valid (piksel persegi)
MIN_AREA = 500

def get_mask(hsv_frame, lower, upper):
    """Buat binary mask untuk rentang HSV yang diberikan."""
    mask = cv2.inRange(hsv_frame, lower, upper)
    # Morphological opening: hilangkan noise kecil
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.erode(mask, kernel, iterations=2)
    mask = cv2.dilate(mask, kernel, iterations=2)
    return mask

def find_largest_contour(mask):
    """Temukan contour terbesar dari mask."""
    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    if not contours:
        return None
    return max(contours, key=cv2.contourArea)

def draw_target(frame, contour, frame_cx, frame_cy):
    """Gambar bounding box, centroid, dan error ke frame."""
    area = cv2.contourArea(contour)
    if area < MIN_AREA:
        return None, None

    # Bounding box
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Centroid via moments
    M = cv2.moments(contour)
    if M["m00"] == 0:
        return None, None
    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])

    # Gambar centroid
    cv2.circle(frame, (cx, cy), 6, (0, 0, 255), -1)

    # Error dari pusat frame
    dx = cx - frame_cx
    dy = cy - frame_cy
    cv2.arrowedLine(frame, (frame_cx, frame_cy), (cx, cy), (255, 0, 0), 2)

    # Label area dan koordinat
    label = f"Area={int(area)} | dx={dx} dy={dy}"
    cv2.putText(frame, label, (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    return dx, dy

# --- Main loop ---
cap = cv2.VideoCapture(0)
```

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    h_frame, w_frame = frame.shape[:2]
    frame_cx, frame_cy = w_frame // 2, h_frame // 2

    # Gambar crosshair tengah frame
    cv2.drawMarker(frame, (frame_cx, frame_cy),
                   (200, 200, 200), cv2.MARKER_CROSS, 20, 1)

    # Pipeline deteksi
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = get_mask(hsv, HSV_LOWER, HSV_UPPER)
    contour = find_largest_contour(mask)

    dx, dy = None, None
    if contour is not None:
        dx, dy = draw_target(frame, contour, frame_cx, frame_cy)

    # Status di pojok kiri atas
    status = f"TARGET DETECTED | dx={dx} dy={dy}" if dx is not None else "NO TARGET"
    color = (0, 255, 0) if dx is not None else (0, 0, 255)
    cv2.putText(frame, status, (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

    # Tampilkan frame utama dan mask
    cv2.imshow("Tracker", frame)
    cv2.imshow("Mask", mask)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Penjelasan langkah pipeline:

Langkah	Fungsi OpenCV	Output
Konversi warna	`cv2.cvtColor(frame, BGR2HSV)`	Frame dalam ruang HSV
Threshold warna	`cv2.inRange(hsv, lower, upper)`	Binary mask (0 atau 255)
Erode	`cv2.erode(mask, kernel, 2)`	Hilangkan noise kecil (piksel terisolasi)
Dilate	`cv2.dilate(mask, kernel, 2)`	Pulihkan ukuran objek setelah erode
Temukan contour	`cv2.findContours(mask, ...)`	List semua kontur pada mask
Hitung centroid	`cv2.moments(contour)`	Koordinat pusat massa objek (cx, cy)
Bounding box	`cv2.boundingRect(contour)`	Kotak pembatas (x, y, w, h)

Mengapa erode → dilate (bukan sebaliknya):
- **Erode** terlebih dahulu: menghilangkan noise kecil dan piksel tepi yang tidak diinginkan
- **Dilate** setelahnya: mengembalikan ukuran objek yang terkikis oleh erode
- Urutan ini disebut **morphological opening** — efektif untuk menghilangkan noise halus tanpa menghilangkan objek utama

Hasil: Pipeline deteksi warna berjalan real-time. Centroid target berhasil dihitung dan error posisi (dx, dy) dari pusat frame berhasil diukur.

---

5. Menangani Warna Merah (Hue Melingkar)

Kegiatan:
Implementasi khusus untuk deteksi warna merah yang memiliki Hue melingkar di rentang 0 dan 179 pada skala OpenCV.

Kode tambahan untuk masking merah:

```
def get_red_mask(hsv_frame):
    """
    Warna merah memiliki Hue di dua ujung rentang (0-10 dan 170-179).
    Perlu dua mask yang digabungkan dengan bitwise OR.
    """
    lower_red1 = np.array([0,   120, 100])
    upper_red1 = np.array([10,  255, 255])
    lower_red2 = np.array([170, 120, 100])
    upper_red2 = np.array([179, 255, 255])

    mask1 = cv2.inRange(hsv_frame, lower_red1, upper_red1)
    mask2 = cv2.inRange(hsv_frame, lower_red2, upper_red2)

    # Gabungkan kedua mask
    mask = cv2.bitwise_or(mask1, mask2)
    return mask
```

Hasil: Deteksi warna merah berhasil dengan menggabungkan dua rentang Hue.

Bagian 4 — Aplikasi Kalibrasi Warna

---

6. Aplikasi Kalibrasi Warna Interaktif

Kegiatan:
Membuat aplikasi kalibrasi interaktif menggunakan trackbar OpenCV untuk menentukan rentang HSV yang tepat bagi target berwarna pada kondisi pencahayaan tertentu. Nilai HSV hasil kalibrasi akan digunakan sebagai konstanta dalam `app_color_tracker.py`.

Kode (`app_color_calibrator.py`):

```
import cv2
import numpy as np
import json
import os

WINDOW_CTRL  = "Kalibrasi Warna — Kontrol HSV"
WINDOW_ORIG  = "Original"
WINDOW_MASK  = "Mask"
```

```
WINDOW_RESULT = "Hasil Masking"
OUTPUT_FILE  = "color_config.json"

def nothing(x):
    pass

def create_trackbars():
    cv2.namedWindow(WINDOW_CTRL, cv2.WINDOW_NORMAL)
    cv2.resizeWindow(WINDOW_CTRL, 400, 300)

    # Trackbar Lower HSV
    cv2.createTrackbar("H Min", WINDOW_CTRL,   0, 179, nothing)
    cv2.createTrackbar("S Min", WINDOW_CTRL,  50, 255, nothing)
    cv2.createTrackbar("V Min", WINDOW_CTRL,  50, 255, nothing)
    # Trackbar Upper HSV
    cv2.createTrackbar("H Max", WINDOW_CTRL, 179, 179, nothing)
    cv2.createTrackbar("S Max", WINDOW_CTRL, 255, 255, nothing)
    cv2.createTrackbar("V Max", WINDOW_CTRL, 255, 255, nothing)

def get_trackbar_values():
    h_min = cv2.getTrackbarPos("H Min", WINDOW_CTRL)
    s_min = cv2.getTrackbarPos("S Min", WINDOW_CTRL)
    v_min = cv2.getTrackbarPos("V Min", WINDOW_CTRL)
    h_max = cv2.getTrackbarPos("H Max", WINDOW_CTRL)
    s_max = cv2.getTrackbarPos("S Max", WINDOW_CTRL)
    v_max = cv2.getTrackbarPos("V Max", WINDOW_CTRL)
    return (
        np.array([h_min, s_min, v_min]),
        np.array([h_max, s_max, v_max])
    )

def save_config(lower, upper, label="target"):
    config = {
        label: {
            "hsv_lower": lower.tolist(),
            "hsv_upper": upper.tolist()
        }
    }
    # Merge dengan config yang sudah ada
    if os.path.exists(OUTPUT_FILE):
        with open(OUTPUT_FILE, "r") as f:
            existing = json.load(f)
        existing.update(config)
        config = existing

    with open(OUTPUT_FILE, "w") as f:
        json.dump(config, f, indent=2)
    print(f"Konfigurasi disimpan ke {OUTPUT_FILE}: lower={lower.tolist()} upper={upper.tolist()}")

def load_config(label="target"):
    if not os.path.exists(OUTPUT_FILE):
        return None, None
    with open(OUTPUT_FILE, "r") as f:
        config = json.load(f)
    if label not in config:
        return None, None
    lower = np.array(config[label]["hsv_lower"])
```

```
    upper = np.array(config[label]["hsv_upper"])
    return lower, upper

create_trackbars()

# Load konfigurasi sebelumnya jika ada
prev_lower, prev_upper = load_config()
if prev_lower is not None:
    cv2.setTrackbarPos("H Min", WINDOW_CTRL, int(prev_lower[0]))
    cv2.setTrackbarPos("S Min", WINDOW_CTRL, int(prev_lower[1]))
    cv2.setTrackbarPos("V Min", WINDOW_CTRL, int(prev_lower[2]))
    cv2.setTrackbarPos("H Max", WINDOW_CTRL, int(prev_upper[0]))
    cv2.setTrackbarPos("S Max", WINDOW_CTRL, int(prev_upper[1]))
    cv2.setTrackbarPos("V Max", WINDOW_CTRL, int(prev_upper[2]))
    print("Konfigurasi sebelumnya dimuat.")

cap = cv2.VideoCapture(0)

print("=" * 50)
print("Kalibrasi Warna HSV")
print("  S = simpan konfigurasi ke color_config.json")
print("  Q = keluar tanpa menyimpan")
print("=" * 50)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lower, upper = get_trackbar_values()

    # Buat mask
    mask = cv2.inRange(hsv, lower, upper)

    # Morphological cleaning
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask_clean = cv2.erode(mask, kernel, iterations=1)
    mask_clean = cv2.dilate(mask_clean, kernel, iterations=1)

    # Terapkan mask ke frame asli
    result = cv2.bitwise_and(frame, frame, mask=mask_clean)

    # Tampilkan nilai HSV di piksel tengah sebagai panduan
    cy, cx = frame.shape[0] // 2, frame.shape[1] // 2
    hsv_center = hsv[cy, cx]
    info = f"Pusat frame HSV: H={hsv_center[0]} S={hsv_center[1]} V={hsv_center[2]}"
    cv2.putText(frame, info, (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1)

    # Gambar crosshair di tengah
    cv2.drawMarker(frame, (cx, cy), (0, 255, 255), cv2.MARKER_CROSS, 20, 1)

    # Tampilkan rentang aktif
    range_info = f"Lower HSV: {lower.tolist()}  |  Upper HSV: {upper.tolist()}"
    cv2.putText(result, range_info, (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)
```

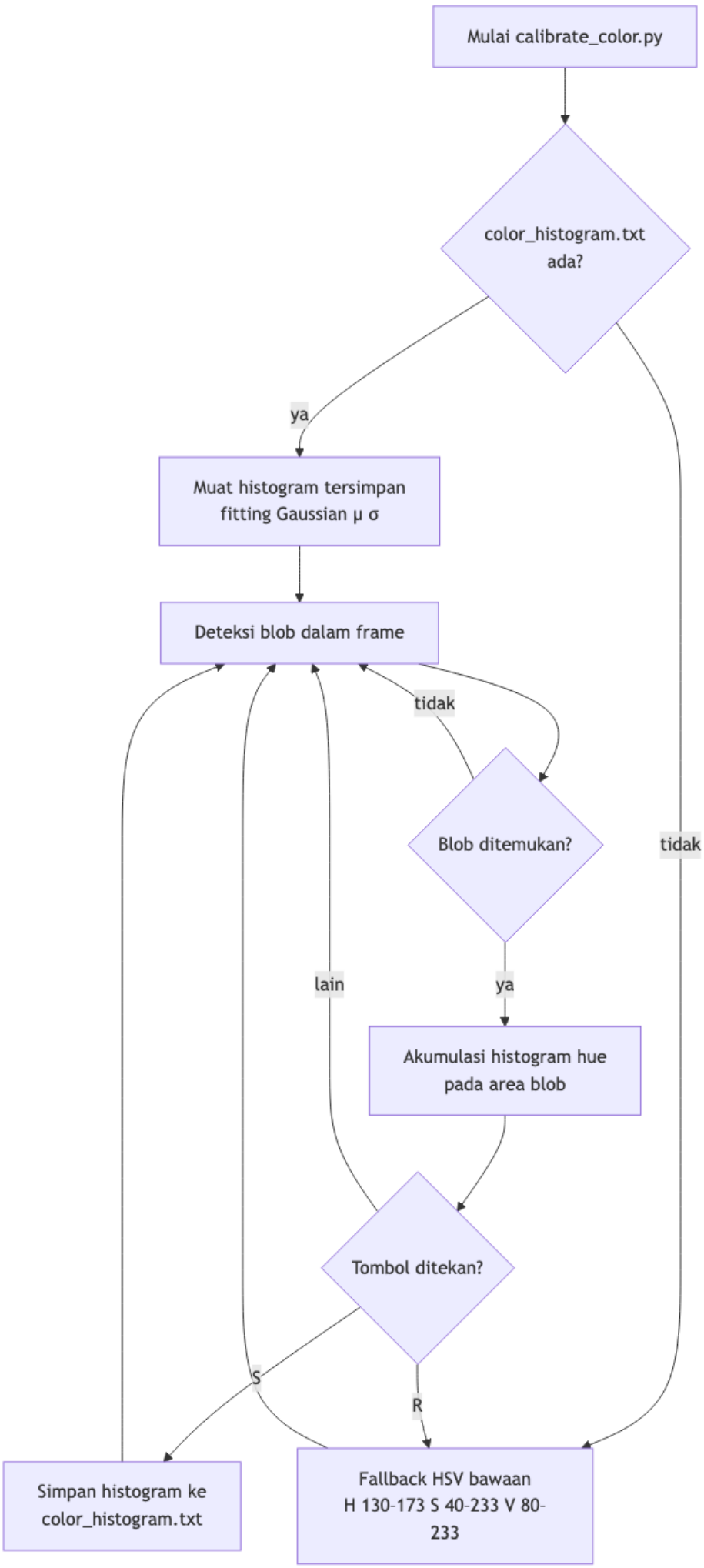
```
    cv2.imshow(WINDOW_ORIG,   frame)
    cv2.imshow(WINDOW_MASK,   mask_clean)
    cv2.imshow(WINDOW_RESULT, result)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('s'):
        save_config(lower, upper)
    elif key == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Cara menggunakan hasil kalibrasi di tracker:

```
import json
import numpy as np

def load_color_config(label="target", path="color_config.json"):
    with open(path) as f:
        cfg = json.load(f)
    return (
        np.array(cfg[label]["hsv_lower"]),
        np.array(cfg[label]["hsv_upper"])
    )

# Di app_color_tracker.py — ganti konstanta hardcode dengan:
HSV_LOWER, HSV_UPPER = load_color_config("target")
```

Contoh output `color_config.json`**:**

```
{
  "target": {
    "hsv_lower": [10, 120, 100],
    "hsv_upper": [25, 255, 255]
  }
}
```


Mulai calibrate_color.py
color_histogram.txt ada?
ya
tidak
Muat histogram tersimpan fitting Gaussian μ σ
Deteksi blob dalam frame
tidak
Blob ditemukan?
lain
ya
Akumulasi histogram hue pada area blob
Tombol ditekan?
S
R
Simpan histogram ke color_histogram.txt
Fallback HSV bawaan H 130-173 S 40-233 V 80-233

Prosedur kalibrasi yang direkomendasikan:

Langkah	Tindakan
1	Tempatkan target di depan kamera pada kondisi pencahayaan aktual
2	Arahkan crosshair tengah frame ke objek target
3	Baca nilai H, S, V yang ditampilkan di layar
4	Set H Min/Max ±10 dari nilai H target
5	Turunkan S Min hingga mask mencakup seluruh objek, tapi tidak background
6	Turunkan V Min jika area bayangan objek tidak terdeteksi
7	Verifikasi: mask hanya menutupi target, background bersih
8	Tekan **S** untuk menyimpan ke `color_config.json`

Troubleshooting kalibrasi:

Masalah	Kemungkinan Penyebab	Solusi
Mask terlalu banyak noise	S Min atau V Min terlalu rendah	Naikkan S Min ke ≥100
Objek tidak terdeteksi	Rentang Hue terlalu sempit	Perlebar H Min dan H Max masing-masing ±5
Mask hilang di area bayangan	V Min terlalu tinggi	Turunkan V Min ke 50–70
Background ikut termasking	Warna objek mirip background	Pertimbangkan pencahayaan yang lebih terkontrol

Hasil: Aplikasi kalibrasi warna interaktif berhasil dibuat. Rentang HSV target dapat ditentukan secara visual dan disimpan ke `color_config.json` untuk digunakan oleh modul tracker.

Desain Model UAV di X-Plane Plane Maker

Selain pengembangan modul drone-seeker, pada hari ini juga dilakukan pemodelan tiga varian UAV — **Satria Nano**, **Satria Talon**, dan **Satria Delta** — di **X-Plane Plane Maker** sebagai dasar simulasi HITL. Plane Maker digunakan untuk mendefinisikan geometri sayap, airfoil, titik CG, dan parameter propulsi secara iteratif sehingga menghasilkan model aerodinamis yang realistis sebelum diuji terbang.

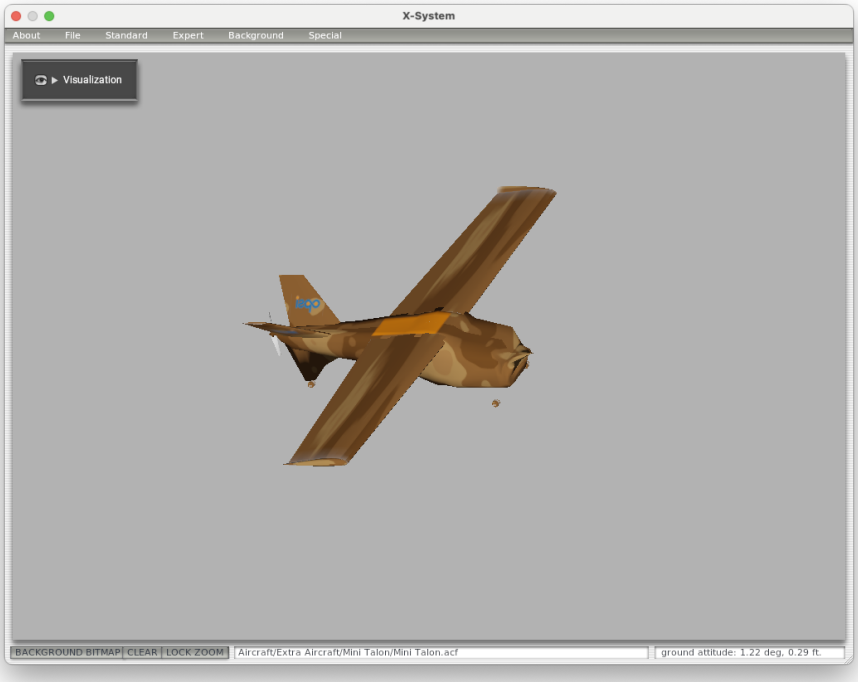
Satria Nano

UAV fixed-wing mini dengan sayap tapered NACA 2412 dan ekor V-tail. Varian terkecil dalam keluarga Satria, dirancang untuk misi ringan dengan payload minimal dan kemudahan pengoperasian.

Parameter	Nilai
Airfoil	**NACA 2412**
Wingspan	**92.66 cm**
Semi-length	**1.52 ft (46.33 cm)**
Wing area	0.1843 m²
Aspect ratio	4.68
Konfigurasi ekor	V-tail twin fin (45°)
Empty weight	**2.20 lb (0.998 kg)**
MTOW	**3.50 lb (1.588 kg)**
Payload	590 g
CG nominal	**0.63 ft = 5.87 cm dari LE MAC**
Wing loading	84.8 N/m²
Stall speed	**9.61 m/s (34.6 km/h)**
Cruise speed	**15.73 m/s (56.6 km/h)**
L/D max	**10.76**
Cruise power	38 W
Endurance	**~45 mnt**
Range one-way	**~1 km**

Parameter	Nilai
Motor	2212, KV 979–1100 @ 3S
Propeller	APC 9×4.5
ESC	30 A BLHeli_32
Baterai	4S 2200 mAh / 3S 3000 mAh

Satria Talon



UAV fixed-wing konvensional berukuran medium dengan sayap tapered NACA 2412 dan ekor V-tail. Dirancang untuk misi endurance jarak jauh dengan payload kamera atau muatan misi.

Parameter	Nilai
Airfoil	**NACA 2412**
Wingspan	**~129.8 cm**
Semi-length	**2.13 ft (64.92 cm)**
Wing area	0.2731 m^2
Aspect ratio	6.17
Konfigurasi ekor	V-tail twin fin (45°)
Empty weight	**3.00 lb (1.361 kg)**
MTOW	**5.25 lb (2.381 kg)**
Payload (4S 8Ah)	571 g
CG nominal	**0.92 ft = 5.87 cm dari LE MAC**
Wing loading	85.5 N/m^2
Stall speed	**9.65 m/s (34.7 km/h)**
Cruise speed	**14.74 m/s (53.1 km/h)**
L/D max	**12.37**
Cruise power	46 W
Endurance	**~131 mnt**
Range one-way	**~116 km**
Radius RTH	**~49–52 km**
Motor	2216, KV 555–600 @ 4S
Propeller	APC 11×5.5E
ESC	40 A BLHeli_32
Baterai	4S 8000 mAh

Satria Delta

UAV konfigurasi delta wing — sayap menyatu tanpa ekor konvensional. Model ini ditambahkan di X-Plane Plane Maker sebagai varian eksperimental untuk misi kecepatan tinggi dan manuverabilitas. Spesifikasi detail masih dalam tahap finalisasi.

Ringkasan Kegiatan

No	Kegiatan	Status
1	Pengenalan OpenCV — konsep dasar, ruang warna, instalasi	✅ Selesai
2	Aplikasi Imshow — tampilkan gambar statis dan feed kamera	✅ Selesai
3	Aplikasi BGR→HSV — visualisasi konversi ruang warna	✅ Selesai
4	Aplikasi Deteksi Objek — pipeline masking, contour, centroid	✅ Selesai
5	Penanganan warna merah (Hue melingkar)	✅ Selesai
6	Aplikasi Kalibrasi Warna — trackbar interaktif + simpan JSON	✅ Selesai
7	Pemodelan Satria Nano di X-Plane Plane Maker	✅ Selesai
8	Pemodelan Satria Talon di X-Plane Plane Maker	✅ Selesai
9	Pemodelan Satria Delta di X-Plane Plane Maker	✅ Selesai

Logbook dibuat: 24 April 2026 | Penelitian OPSI 2026 — SMA Swasta Alfa Centauri